

Kubernetes Configuration Improvement Guide

Audit | Critical Fixes | Best Practices
for k8s/k8s Monorepo

Repository: devops/k8s/k8s
6 active apps (omniroute/umami removed)

VPS A (2GB/2CPU): mazanoke, mbakmegumi, ekel-backend, goatcounter
+ Traefik + cert-manager (shell script deploy)

VPS B (4GB/2CPU): ai-assistant, ryuko-matoi-go + ArgoCD

Table of Contents

- 1. 1. Cluster Architecture
- 2. 2. Phase 1: Critical Fixes
- 3. 3. Phase 2: Network & Security
- 4. 4. Phase 3: Best Practices
- 5. 5. Full Config Examples
- 6. 6. Priority & Execution Matrix
- 7. 7. Appendix: Verification Commands

1. Cluster Architecture

VPS A - Lightweight Apps (2GB/2CPU)

Deploy: Shell script (deploy-k8s.sh) + envsubst

Apps: mazonoke (nginx static), mbakmegumi (Astro/nginx), ekel-backend (Go/Echo), goatcounter (Go/SQLite)

Infra: Traefik + cert-manager

Free RAM: ~700MB-1GB after infra pods

VPS B - Stateful/Bot Apps (4GB/2CPU)

Deploy: ArgoCD (GitOps)

Apps: ai-assistant (Telegram bot), ryuko-matoi-go (WhatsApp bot)

Infra: ArgoCD server

Free RAM: ~1.5-2GB

Removed

omniroute (Next.js+Redis), umami (analytics+PostgreSQL)

IMPORTANT: All resource values are calibrated for these VPS specs.

2. Phase 1: Critical Fixes

2a. Typo: argocd/secret.yaml

CRITICAL

Line 2: "king: Secret" should be "kind: Secret". Resource WILL fail to apply.

```
apiVersion: v1
kind: Secret      # was "king: Secret"
metadata:
  name: argocd-env
  namespace: argocd
type: Opaque
stringData:
  DOMAIN: ${DOMAIN}
  EMAIL: ${EMAIL}
```

>> Verify: `kubectl apply --dry-run=client -f argocd/secret.yaml`

2b. Missing securityContext (4 apps)

CRITICAL

Apps running without security hardening:

ekel-backend/deployment.yaml

Issue: No pod-level or container-level securityContext
Fix: Add runAsNonRoot, seccompProfile, drop ALL caps

ryuko-matoi-go/deployment.yaml

Issue: No securityContext + uses hostPath
Fix: Add securityContext + switch to PVC

ai-assistant/deployment.yaml

Issue: No securityContext at pod/init/main levels
Fix: Add at all three levels

goatcounter/deployment.yaml

Issue: No container securityContext + no resources
Fix: Add resources + securityContext

Security Context Template:

```
# Pod-level (under spec:)
securityContext:
  runAsNonRoot: true
  runAsUser: 65534
  runAsGroup: 65534
  fsGroup: 65534
  fsGroupChangePolicy: OnRootMismatch
  seccompProfile:
    type: RuntimeDefault

# Container-level (under containers[name]):
securityContext:
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities:
    drop:
      - ALL
```

>> Note: set `readOnlyRootFilesystem:false` if app needs `/tmp` write access

2c. ryuko-matoi-go: hostPath -> PVC

HIGH

`hostPath` pins pod to one node. PVC (1Gi) already exists but NOT used.

```
# Before (in deployment volumes:)
- name: app-data
  hostPath:
    path: /data/ryuko-matoi
    type: DirectoryOrCreate

# After
- name: app-data
  persistentVolumeClaim:
    claimName: ryuko-matoi-data
```

2d. Missing ryuko-matoi-go/secret.yaml

HIGH

Deployment references "ryuko-matoi-env" but no `secret.yaml` exists.
Public API URLs should NOT be in secrets - move to `ConfigMap`.

secret.yaml (CREATE NEW - actual secrets only):

```

apiVersion: v1
kind: Secret
metadata:
  name: ryuko-matoi-env
type: Opaque
stringData:
  REMOVE_BG_API_KEY: ${REMOVE_BG_API_KEY}
  AI_API_KEY: ${AI_API_KEY}
  AI_PROVIDER: ${AI_PROVIDER}
  AI_MODEL: ${AI_MODEL}
  WHATSAPP_SESSION_PATH: ${WHATSAPP_SESSION_PATH}
  WHATSAPP_DEVICE_NAME: ${WHATSAPP_DEVICE_NAME}
  WHATSAPP_DATABASE_DIALECT: ${WHATSAPP_DATABASE_DIALECT}
  WHATSAPP_EVENT_BUFFER_SIZE: ${WHATSAPP_EVENT_BUFFER_SIZE}
  OCR_PROVIDER: ${OCR_PROVIDER}
  OCR_LANGUAGE: ${OCR_LANGUAGE}
  OCR_BINARY: ${OCR_BINARY}

```

configmap.yaml (CREATE NEW - public URLs + config):

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: ryuko-matoi-config
data:
  APP_NAME: "Ryuko Matoi"
  APP_ENV: "production"
  TZ: "Asia/Jakarta"
  LOG_LEVEL: "info"
  LOG_DIR: "/app/logs"
  BRAT_FONT_PATH: "/app/assets/Brat.ttf"
  JOKES_API_URL: "https://official-joke-api.appspot.com/random_joke"
  ANIME_QUOTE_API_URL: "https://animechan.xyz/api/random"
  QURAN_API_URL: "https://api.alquran.cloud/v1/ayah/random"
  DOA_API_URL: "https://api.myquran.com/v2/doa/semua"
  IMAGE_API_URL: "https://api.unsplash.com/photos/random"

```

In deployment.yaml, replace secretKeyRef with configMapKeyRef for public URLs:

```

# Before
- name: JOKES_API_URL
  valueFrom:
    secretKeyRef:
      name: ryuko-matoi-env
      key: JOKES_API_URL

# After
- name: JOKES_API_URL
  valueFrom:
    configMapKeyRef:
      name: ryuko-matoi-config
      key: JOKES_API_URL

```

2e. goatcounter: missing resources + service

MEDIUM

No resource requests/limits = no QoS. Service missing labels/type/port-name.

```
# Add to goatcounter container in deployment.yaml:
resources:
  requests:
    cpu: 50m
    memory: 64Mi
  limits:
    cpu: 300m
    memory: 128Mi

# Fix goatcounter/services.yaml:
apiVersion: v1
kind: Service
metadata:
  name: goatcounter
  labels:
    app: goatcounter
spec:
  type: ClusterIP
  selector:
    app: goatcounter
  ports:
    - port: 80
      targetPort: 3000
      protocol: TCP
      name: http
```

3. Phase 2: Network & Security

3a. NetworkPolicy

NEW

Only removed umami had NetworkPolicies. Add for remaining apps:

goatcounter (SQLite - allow Traefik only):

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: goatcounter-allow-traefik
spec:
  podSelector:
    matchLabels:
      app: goatcounter
  policyTypes: [Ingress]
  ingress:
    - from:
      - namespaceSelector: {}
        podSelector:
          matchLabels:
            app.kubernetes.io/name: traefik
  ports:
    - protocol: TCP
      port: 3000
```

ai-assistant (Telegram bot - egress to internet):

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: ai-assistant-restrict
spec:
  podSelector:
    matchLabels:
      app: ai-assistant
  policyTypes: [Egress]
  egress:
    - to:
      - namespaceSelector: {}
        podSelector: {}
      ports: [{protocol: UDP, port: 53}] # DNS
    - to:
      - ipBlock:
          cidr: 0.0.0.0/0
          except: [10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16]
      ports: [{protocol: TCP, port: 443}] # Telegram + AI APIs
```

ryuko-matoi-go (WhatsApp bot - egress only):


```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: ryuko-matoi-restrict
spec:
  podSelector:
    matchLabels:
      app: ryuko-matoi
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector: {}
        - podSelector: {}
      ports: [{protocol: UDP, port: 53}] # DNS
    - to:
        - ipBlock:
            cidr: 0.0.0.0/0
            except: [10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16]
      ports: [{protocol: TCP, port: 443}, {protocol: TCP, port: 5222}]

```

3b. Traefik Security Middleware

NEW

No security headers or rate limiting on any ingress.

```

apiVersion: traefik.io/v1alpha1
kind: Middleware
metadata:
  name: security-headers
spec:
  headers:
    frameDeny: true
    browserXssFilter: true
    contentTypeNosniff: true
    referrerPolicy: "strict-origin-when-cross-origin"
    permissionsPolicy: "camera=(), microphone=(), geolocation=()"
---
apiVersion: traefik.io/v1alpha1
kind: Middleware
metadata:
  name: rate-limit
spec:
  rateLimit:
    average: 50
    burst: 30

```

Attach to ingress via annotation:

```

metadata:
  annotations:
    traefik.ingress.kubernetes.io/router.middlewares: >-
      default-security-headers@kubernetescrd,
      default-rate-limit@kubernetescrd

```

3c. Secrets Management

NEW

Plain YAML secrets in git = security gap. Use Sealed Secrets:

```
# Install (once per cluster):  
kubectl apply -f https://github.com/bitnami-labs/sealed-secrets/releases/download/v0.27.0/controller.yaml  
  
# Encrypt existing secrets:  
kubeseal -o yaml < secret.yaml > sealed-secret.yaml  
# sealed-secret.yaml is safe to commit
```

4. Phase 3: Best Practice Improvements

4a. Kustomize (replace envsubst)

NEW

Current: .env -> envsubst -> kubectl apply. No diff, no rollback, no env separation.

Kustomize adds overlays without rewriting YAML:

```
k8s/base/mazanoke/
deployment.yaml # no ${VAR}, uses ConfigMap
service.yaml
ingress.yaml
kustomization.yaml

k8s/overlays/production-vps-a/
kustomization.yaml # namespace, replicas, image tags
```

base/mazanoke/kustomization.yaml:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- deployment.yaml
- service.yaml
- ingress.yaml
configMapGenerator:
- name: mazanoke-config
  literals:
  - DOMAIN=mazanoke.haikel.my.id
```

overlays/production-vps-a/kustomization.yaml:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: apps
resources:
- ../../base/mazanoke
- ../../base/mbakmegumi
- ../../base/ekel-backend
- ../../base/goatcounter
images:
- name: haikelz/mazanoke
  newTag: v1.2.0
```

Deploy: kubectl apply -k overlays/production-vps-a/

4b. ArgoCD App of Apps (VPS B)

NEW

Single root Application manages all ArgoCD apps. Push = auto-deploy.

```
# k8s/argocd/app-of-apps.yaml (CREATE NEW)
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: apps-root
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/haikelz/devops.git
    targetRevision: master
    path: k8s/argocd/apps
  destination:
    server: https://kubernetes.default.svc
    namespace: argocd
  syncPolicy:
    automated:
      prune: true
      selfHeal: true

# k8s/argocd/apps/ai-assistant.yaml (CREATE NEW)
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: ai-assistant
  namespace: argocd
  finalizers: [resources-finalizer.argocd.argoproj.io]
spec:
  project: default
  source:
    repoURL: https://github.com/haikelz/ai-assistant
    targetRevision: master
    path: k8s
  destination:
    server: https://kubernetes.default.svc
    namespace: bots
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
  retry:
    limit: 3
    backoff: {duration: 5s, factor: 2, maxDuration: 1m}
```

4c. Namespace Isolation + ResourceQuota

NEW

All apps in "default" namespace - create logical groups:

```
kubectl create ns apps    # mazanoke, mbakmegumi, ekel-backend, goatcounter
kubectl create ns bots    # ai-assistant, ryuko-matoi-go
```

ResourceQuota - VPS A (2GB):

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: vps-a-quota
  namespace: apps
spec:
  hard:
    requests.cpu: "1500m"
    requests.memory: 1.5Gi
    limits.cpu: "3"
    limits.memory: 2.5Gi
    persistentvolumeclaims: "5"
---
apiVersion: v1
kind: LimitRange
metadata:
  name: vps-a-limits
  namespace: apps
spec:
  limits:
    - type: Container
      default: {cpu: 200m, memory: 256Mi}
      defaultRequest: {cpu: 50m, memory: 64Mi}
      max: {cpu: "1", memory: 512Mi}
      min: {cpu: 20m, memory: 32Mi}
```

4d. PodDisruptionBudget

NEW

Single-replica stateful apps need PDB to survive node maintenance:

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: goatcounter-pdb
  namespace: apps
spec:
  maxUnavailable: 0
  selector:
    matchLabels:
      app: goatcounter
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: ryuko-matoi-pdb
  namespace: bots
spec:
  maxUnavailable: 0
  selector:
    matchLabels:
      app: ryuko-matoi
```

4e. Image Tag Pinning

NEW

:latest + Always pull = different image on every restart.

```
# Before
image: haikelilham/ryuko-matoi-go:latest
imagePullPolicy: Always

# After
image: haikelilham/ryuko-matoi-go:v1.5.2
imagePullPolicy: IfNotPresent
```

4f. Recreate Strategy for Stateful Apps

NEW

Goatcounter (SQLite), ryuko-matoi-go (WhatsApp session lock) cannot have 2 replicas.
Use Recreate instead of default RollingUpdate:

```
spec:
  strategy:
    type: Recreate
```

4g. Priority Classes

NEW

Traefik + cert-manager killed LAST under memory pressure:

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: infra-high
value: 1000000
globalDefault: false
description: "Infrastructure"
---
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: app-default
value: 1000
globalDefault: true
description: "Standard workloads"
```

4h. Database Backup CronJob

NEW

SQLite apps have zero backup. Add daily CronJob:

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: goatcounter-backup
  namespace: apps
spec:
  schedule: "0 2 * * *"
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: backup
              image: alpine:3.20
              command: [sh, -c]
              args:
                - cp /data/goatcounter.sqlite3 /backup/goatcounter-$(date +%Y%m%d).sqlite3
              volumeMounts:
                - {name: data, mountPath: /data}
                - {name: backup, mountPath: /backup}
          volumes:
            - {name: data, persistentVolumeClaim: {claimName: goatcounter-data}}
            - {name: backup, persistentVolumeClaim: {claimName: goatcounter-backup}}
```

5. Full Config Examples (Resource-Adjusted)

5a. ekel-backend Deployment (VPS A, 2GB)


```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ekel-backend
  namespace: apps
  labels:
    app: ekel-backend
spec:
  replicas: 1
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 0
      maxUnavailable: 1
  selector:
    matchLabels:
      app: ekel-backend
  template:
    spec:
      terminationGracePeriodSeconds: 60
      securityContext:
        runAsNonRoot: true
        runAsUser: 65534
        runAsGroup: 65534
        fsGroup: 65534
        fsGroupChangePolicy: OnRootMismatch
        seccompProfile:
          type: RuntimeDefault
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app: ekel-backend
      containers:
        - name: ekel-backend
          image: ${IMAGE}
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 9090
              protocol: TCP
              name: http
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: false
            capabilities:
              drop: [ALL]
          resources:
            requests:
              cpu: 50m
              memory: 64Mi
            limits:
              cpu: 300m
              memory: 128Mi
      env:
        - {name: PORT, value: "9090"}
        - {name: APP_ENV, value: "production"}
        - {name: APP_DEBUG, value: "false"}
        - name: WAKATIME_API_KEY
          valueFrom:
            secretKeyRef:
              name: ekel-backend-env
              key: WAKATIME_API_KEY
```

```
- name: TURSO_AUTH_TOKEN
  valueFrom:
    secretKeyRef:
      name: ekel-backend-env
      key: TURSO_AUTH_TOKEN
- name: TURSO_DATABASE_URL
  valueFrom:
    secretKeyRef:
      name: ekel-backend-env
      key: TURSO_DATABASE_URL
- name: JWT_SECRET
  valueFrom:
    secretKeyRef:
      name: ekel-backend-env
      key: JWT_SECRET
livenessProbe:
  httpGet: {path: /ping, port: 9090}
  initialDelaySeconds: 10
  periodSeconds: 30
readinessProbe:
  httpGet: {path: /ping, port: 9090}
  initialDelaySeconds: 5
  periodSeconds: 10
startupProbe:
  httpGet: {path: /ping, port: 9090}
  initialDelaySeconds: 5
  periodSeconds: 10
  failureThreshold: 12
```

5b. goatcounter Deployment (VPS A, 2GB)

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: goatcounter
  namespace: apps
spec:
  replicas: 1
  strategy:
    type: Recreate      # SQLite lock
  selector:
    matchLabels:
      app: goatcounter
  template:
    spec:
      terminationGracePeriodSeconds: 60
      securityContext:
        runAsNonRoot: true
        runAsUser: 65534
        runAsGroup: 65534
        fsGroup: 65534
        fsGroupChangePolicy: OnRootMismatch
        seccompProfile:
          type: RuntimeDefault
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app: goatcounter
      enableServiceLinks: false
      initContainers:
        - name: init-goatcounter
          image: ${IMAGE}
          command: [sh, -c]
          args:
            - |
              if [ ! -f /data/goatcounter.sqlite3 ]; then
                goatcounter db create site -createdb          -db sqlite3+/data/goatcounter.sqlite3      -vhost
              fi
      envFrom:
        - secretRef:
            name: goatcounter-env
      volumeMounts:
        - {name: goatcounter-data, mountPath: /data}
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: false
        capabilities: {drop: [ALL]}
      containers:
        - name: goatcounter
          image: ${IMAGE}
          args:
            - serve
            - -listen
            - :3000
            - -tls
            - none
            - -db
            - sqlite3+/data/goatcounter.sqlite3
            - -automigrate
            - -email-from
            - ${EMAIL}
          ports:

```

```
- containerPort: 3000
resources:
  requests:
    cpu: 50m
    memory: 64Mi
  limits:
    cpu: 300m
    memory: 128Mi
volumeMounts:
  - {name: goatcounter-data, mountPath: /data}
securityContext:
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: false
  capabilities: {drop: [ALL]}
volumes:
  - name: goatcounter-data
    persistentVolumeClaim:
      claimName: goatcounter-data
```

5c. ai-assistant Deployment (VPS B, 4GB)

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-assistant
  namespace: bots
  labels:
    app: ai-assistant
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app: ai-assistant
  template:
    spec:
      terminationGracePeriodSeconds: 30
      securityContext:
        runAsNonRoot: true
        runAsUser: 1000
        runAsGroup: 1000
        fsGroup: 1000
        fsGroupChangePolicy: OnRootMismatch
        seccompProfile:
          type: RuntimeDefault
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app: ai-assistant
      volumes:
        - name: picoclaw-data
          persistentVolumeClaim:
            claimName: ai-assistant-data
      initContainers:
        - name: initialize-workspace
          image: ${IMAGE}
          imagePullPolicy: IfNotPresent
          command: [/bin/sh, -ec]
          args:
            - |
              mkdir -p /root/.picoclaw
              jq -r '.channel_list.telegram.allow_from = [env.TELEGRAM_USER_ID]' /seed/config.json > /root/.picoclaw/config.json
              mv /root/.picoclaw/config.json.tmp /root/.picoclaw/config.json
              chmod 600 /root/.picoclaw/config.json
      envFrom:
        - secretRef:
            name: ai-assistant-env
      volumeMounts:
        - {name: picoclaw-data, mountPath: /root/.picoclaw}
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: false
        capabilities: {drop: [ALL]}
        - name: initialize-paycheck-reminder
          image: ${IMAGE}
          imagePullPolicy: IfNotPresent
          command: [/bin/sh, -ec]
          args:
            - |
              if ! picoclaw cron list | grep -Fq monthly-paycheck-reminder; then
                picoclaw cron add --name monthly-paycheck-reminder --cron "0 9 28 * *" --channel telegram --message "Paycheck re

```

```
    fi
  volumeMounts:
    - {name: picoclaw-data, mountPath: /root/.picoclaw}
  securityContext:
    allowPrivilegeEscalation: false
    readOnlyRootFilesystem: false
    capabilities: {drop: [ALL]}
containers:
- name: picoclaw
  image: ${IMAGE}
  imagePullPolicy: IfNotPresent
  ports:
    - {name: http, containerPort: 18790}
  readinessProbe:
    httpGet: {path: /ready, port: http}
    initialDelaySeconds: 5
    periodSeconds: 10
  livenessProbe:
    httpGet: {path: /health, port: http}
    initialDelaySeconds: 20
    periodSeconds: 20
  resources:
    requests:
      cpu: 100m
      memory: 256Mi
    limits:
      cpu: 1000m
      memory: 1Gi
  volumeMounts:
    - {name: picoclaw-data, mountPath: /root/.picoclaw}
  securityContext:
    allowPrivilegeEscalation: false
    readOnlyRootFilesystem: false
    capabilities: {drop: [ALL]}
```


6. Priority & Execution Matrix

Immediate (Week 1)

1. Fix argocd/secret.yaml typo - 1 min
2. Add securityContext to ekel-backend - 15 min
3. Add securityContext to ai-assistant - 15 min
4. Add securityContext to goatcounter + add resources - 10 min
5. Create ryuko-matoi-go/secret.yaml + configmap.yaml - 10 min
6. Switch ryuko-matoi-go hostPath -> PVC - 5 min
7. Fix goatcounter service format - 2 min

Short Term (Week 2-4)

8. Create NetworkPolicy for goatcounter, ai-assistant, ryuko-matoi-go - 30 min
9. Add Traefik security middleware - 15 min
10. Create namespaces (apps, bots) - 5 min
11. Add ResourceQuota + LimitRange - 10 min
12. Change strategy to Recreate for goatcounter + ryuko-matoi-go - 5 min
13. Add PDB for goatcounter + ryuko-matoi-go - 5 min
14. Set up Sealed Secrets - 30 min

Medium Term (Month 2-3)

15. Migrate VPS A to Kustomize (replace envsubst) - 3 hours
16. ArgoCD App of Apps for VPS B - 1 hour
17. Database backup CronJob - 30 min
18. Image tag pinning - 10 min per app
19. Priority Classes - 10 min

Long Term

20. Monitoring (Prometheus + Grafana) - be mindful of 2GB limit
21. Kyverno policy enforcement
22. HPA for scalable apps
23. Full GitOps for VPS A via ArgoCD (if RAM allows)

7. Appendix: Verification Commands

Pre-Deploy Checks

```
# Validate YAML syntax
kubectl apply --dry-run=client -f deployment.yaml

# Check security context
kubectl get pod <pod> -o jsonpath="{.spec.containers[].securityContext}" | jq

# Check if running as root
kubectl exec <pod> -- id
```

Post-Deploy Verification

```
# Check pod security
kubectl get pods -o json | jq ".items[].spec.containers[].securityContext"

# Verify NetworkPolicy
kubectl get networkpolicy -A

# Check resource usage
kubectl top pods -A
kubectl top nodes

# Check quota usage
kubectl describe resourcequota -A

# Check PDB status
kubectl get pdb -A

# Verify ArgoCD sync
argocd app list

# Check cert-manager
kubectl get certificates -A
kubectl get orders -A
```

Quick Health Check

```
# All-in-one status
kubectl get deploy,svc,ing,pvc,networkpolicy,pdb -A

# Events
kubectl get events --sort-by=.metadata.creationTimestamp | tail -20

# Pod restarts
kubectl get pods -A -o json | jq -r ".items[] | select(.status.containerStatuses[].restartCount > 3) | .metadata.name"
```

Files Created by This Guide

New files to create:

- k8s/k8s/ryuko-matoi-go/secret.yaml
- k8s/k8s/ryuko-matoi-go/configmap.yaml
- k8s/k8s/goatcounter/networkpolicy.yaml
- k8s/k8s/ai-assistant/networkpolicy.yaml

- k8s/k8s/ryuko-matoi-go/networkpolicy.yaml
- k8s/k8s/goatcounter/pdb.yaml
- k8s/k8s/ryuko-matoi-go/pdb.yaml
- k8s/argocd/app-of-apps.yaml
- k8s/argocd/apps/ai-assistant.yaml
- k8s/argocd/apps/ryuko-matoi-go.yaml
- k8s/k8s/shared/priorityclasses.yaml

Files to modify:

- k8s/k8s/argocd/secret.yaml (fix typo)
- k8s/k8s/ekel-backend/deployment.yaml (add securityContext)
- k8s/k8s/goatcounter/deployment.yaml (add resources + securityContext)
- k8s/k8s/goatcounter/services.yaml (add labels/type)
- k8s/k8s/ai-assistant/deployment.yaml (add securityContext)
- k8s/k8s/ryuko-matoi-go/deployment.yaml (hostPath->PVC + securityContext)